MARF🎭: Masking Bundle-Adjusting Neural Radiance Fields

Thomas Jaron-Strugala
Technical University of Berlin
jaron-strugala@campus.tu-berlin.de

Oliver Jan Jarosik
Technical University of Berlin
jarosik@campus.tu-berlin.de

Simon Bonaventura Ertlmaier
Technical University of Berlin
s.ertlmaier@campus.tu-berlin.de

March 28, 2026

***K*eywords** Image Restoration · Image Alignment · Neural Radiance Fields · Mask Generation

1 Introduction

Two-dimensional image alignment is a fundamental process in computer vision and image analysis that involves transforming different images into a common coordinate system. Image alignment is of high importance in image restoration, where aligning potentially obfuscated or occluded fragments of a picture to create an accurate replication may be necessary. Deep learning is used in many state-of-the-art methods that aim to solve this problem. In the three-dimensional domain, there are already several approaches that address image alignment using deep learning, such as Neural Radiance Fields [9]. Another deep learning model that builds on Neural Radiance Fields is Bundle-Adjusting Neural Radiance Fields [8], which reduces the impact of precise image positioning during the learning process. While these are three-dimensional alignment approaches, both can be applied to the two-dimensional case as well. In this paper, we will discuss **MARF**, an approach for two-dimensional image alignment that improves upon the method used by Bundle-Adjusting Neural Radiance Fields, while also considering occlusions that might occur in images.

2 Related Work

Before we explain our proposed model we will discuss the models and tools that are fundamental for various image and 3D reconstruction approaches.

2.1 Neural Radiance Fields

Neural Radiance Fields (NeRF) [9] are using images taken from various camera positions to reconstruct a 3D scene. NeRF consists of a multilayer perceptron (MLP) [3] to act as the volumetric function F_θ to represent one singular scene. This MLP receives as inputs a 3D position $\mathbf{x} = (x, y, z)$ and a 2D viewing direction $\mathbf{d} = (\alpha, \beta)$. It returns the color $\mathbf{c} = (r, g, b)$ and the volume density σ:

$$(\sigma, z) = F_{\theta_1}(\gamma_x(\mathbf{x})), \tag{1}$$

$$\mathbf{c} = F_{\theta_2}(\gamma_d(\mathbf{d}), z), \tag{2}$$

where $\gamma_x(\cdot)$ and $\gamma_d(\cdot)$ are the positional encoding functions applied to $\mathbf{x}$ and $\mathbf{d}$ respectively, and $\theta = (\theta_1, \theta_2)$ are the MLP parameters.

Building the scene requires calculating the color of multiple rays crossing the scene. The mentioned ray is constructed via the linear function

$$\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}, \tag{3}$$

with $\mathbf{o}$ being the center of the camera and $\mathbf{d}$ the correlated viewing direction.

The final pixel color $\widehat{\mathbf{C}}(\mathbf{r})$ is computed by integrating the contributions of all sampled points along the ray using the volume rendering equation:

$$\widehat{\mathbf{C}}(\mathbf{r}) = \sum_{i=1}^{N} T_i(1 - \exp(-\sigma_i\delta_i))\mathbf{c}_i, \tag{4}$$

$$T_i = \exp\left(-\sum_{j=1}^{i-1}\sigma_j\delta_j\right), \tag{5}$$

where $\delta_i = t_{i+1} - t_i$ is the distance between adjacent samples, and T_i is the accumulated transmittance. The term $1-\exp(-\sigma_i\delta_i)$ represents the probability of the ray terminating at $\mathbf{r}(t_i)$, and T_i denotes the accumulated transmittance from the near plane to $\mathbf{r}(t_i)$.

To optimise the MLP parameters, the sum of squared errors between a collection of images $\{I_i\}_{i=1}^{N}$ (with N images) and the corresponding rendered output is minimised. NeRF calculates in advance the amount of camera rays $\{r_{ij}\}$ for each pixel j from the image I_i. All parameters are optimised by minimising the following loss function

$$L = \sum_{ij}\left\|C(r_{ij}) - \hat{C}(r_{ij})\right\|_2^2 \tag{6}$$

where $C(r_{ij})$ is the observed colour of ray j in image I_i and $\hat{C}(r_{ij})$ is the predicted colour of the ray.

NeRF requires precise camera positions and corresponding multi-view images to produce high-quality renderings. If the camera positions are not correct, the NeRF model will have problems reconstructing the scene accurately, resulting in inaccuracies in the rendered images, e.g. artefacts, blurring or incorrect geometry. This significantly reduces the quality of the generated images. Hierarchical volume sampling and positional encoding [7] are two integrated strategies in NeRF to rise the resilience of inaccurate camera positions. Hierarchical Volume Sampling increases rendering performance and quality by dynamically capturing more points when needed. Position encoding improves the ability of the MLP to detect high-frequency information. The input coordinates $\mathbf{x}$ and $\mathbf{d}$ using sine and cosine functions with exponentially increasing frequencies projected into a higher dimensional space. This technique helps to avoid blurring in the final images. The position coding function is defined as follows:

$$\gamma(p) = \left(\sin(2^0\pi p), \cos(2^0\pi p), \ldots, \sin(2^{L-1}\pi p), \cos(2^{L-1}\pi p)\right), \tag{7}$$

where p is a component of $\mathbf{x}$ or $\mathbf{d}$, and L represents the number of frequency bands.

2.2 Bundle-Adjusting Neural Radiance Fields

Unlike NeRF, Bundle-Adjusting Neural Radiance Fields (BARF) [8] does not require precise camera parameters. In case of missing camera positions the location can be approximated via positional encoding. However, this approach can lead to incorrect camera registrations, when different frequencies inside the positional encoding cancel out each other.

To compensate these errors BARF adapts the positional encoding function:

$$\gamma_k(x;\alpha) = w_k(\alpha)\cdot\begin{pmatrix}\cos(2^k\pi x)\\ \sin(2^k\pi x)\end{pmatrix} \tag{8}$$

Starting with broad, less detailed data and subsequently refining it with finer information is known as the "coarse-to-fine" technique [4]. Higher frequency components are progressively added as optimization advances in order to improve picture representation and capture more details.
The weighting function $w_k(\alpha)$ used in BARF regulates the contribution of each frequency component according to the optimization progress, which is indicated by the parameter α.

The weight function $w_k(\alpha)$ is defined by:

$$w_k(\alpha) = \begin{cases} 0 & \text{if } \alpha < k, \\ 1 - \frac{\cos((\alpha-k)\pi)}{2} & \text{if } 0 \leq \alpha - k < 1, \\ 1 & \text{if } \alpha - k \geq 1. \end{cases} \tag{9}$$

During the optimisation process the parameter α is increased steadily. Initially, the k-th frequency component is inactive when α is smaller than k. The weight $w_k(\alpha)$ smoothly goes from 0 to 1, progressively integrating higher frequencies into the encoding as α increases.

The Jacobian of the encoding function, which measures how changes in 3D coordinates affect the positional encoding, is given by:

$$\frac{\partial \gamma_k(x;\alpha)}{\partial x} = w_k(\alpha) \cdot 2^k \pi \cdot \begin{pmatrix} -sin(2^k \pi x) \\ cos(2^k \pi x) \end{pmatrix}. \tag{10}$$

This Jacobian shows that the contribution of higher frequency components to the gradient becomes zero when $w_k(\alpha) = 0$, which lessens the effect of high-frequency fluctuations in the initial phases of optimization.

2.3 Hallucinated Radiance Fields in the Wild

Hallucinated Radiance Fields in the Wild (Ha-NeRF) [1] eliminate the limitations of previous methods by handling dynamic features and changing lighting circumstances to provide renderings that are both occlusion-free and consistent with the view. The appearance hallucination and anti-occlusion are the two primary components introduced by the Ha-NeRF architecture.

Every input picture I_i is processed by a Convolutional Neural Network (CNN) to create an appearance latent vector ℓ_{a_i} for hallucination. Essential visual elements, such as color and lighting, are captured by this vector. Therefore the CNN acts as a encoder E_ϕ for an image. The parameter ϕ represents the photometric postprocessing.

$$\ell_{a_i} = E_\phi(I_i). \tag{11}$$

During rendering, 3D locations and viewing directions are sampled from camera rays. These samples, along with the appearance latent vector ℓ_{a_i}, are fed into a MLP to predict color c and volume density σ at each scene point. The volumetric function F_θ uses positional encodings for location $\gamma(x)$ and viewing direction $\gamma(d)$:

$$(c, \sigma) = F_\theta(\gamma(x), \gamma(d), \ell_{a_i}). \tag{12}$$

The rendered image $\hat{I}_i$ is obtained by integrating these predictions over the sampled camera rays. To ensure that the appearance remains consistent across different views, a view-consistent appearance loss L_v is employed:

$$L_v = \|E_\phi(\hat{I}_i) - \ell_{a_i}\|_1. \tag{13}$$

This loss function penalizes differences between appearance features encoded from different views, ensuring that the appearance vector ℓ_{a_i} remains consistent.
Both the appearance encoder E_ϕ and the radiance field F_θ are jointly optimized. To enhance efficiency, a grid of rays is sampled to create the image $\hat{I}_i$ during training, rather than rendering an entire image. This approach assumes that the global appearance vector is stable after random grid sampling.

To differentiate between static and dynamic elements an image-dependent 2D visibility map for anti-occlusion handling is used. An additional MLP predicts visibility probabilities M_{ij} for each pixel location p_{ij}, given by:

$$M_{ij} = G_\theta(p_{ij}, \ell_{\tau_i}), \tag{14}$$

where G_θ is the MLP for visibility prediction, and ℓ_{τ_i} represents transient features. These probabilities create a visibility mask that highlights static parts of the scene while filtering out transient elements. The anti-occlusion loss

$$L_o = M_{ij}\left|\left|C(r_{ij}) - \hat{C}(r_{ij})\right|\right|_2^2 + \lambda_o \left(1 - M_{ij}\right)^2 \tag{15}$$

controls the balance between reconstruction accuracy and preventing the model from incorrectly marking visible pixels as invisible.

2.4 Synthetic Image Dataset for Alignment and Restoration

We use Synthetic Image Dataset for Alignment and Restoration (SIDAR)[6], a pipeline designed to generate datasets for image alignment tasks. SIDAR uses 3D rendering techniques to simulate the projection of a 2D image onto a plane within a virtual environment. This creates synthetic ground truth data, which we used to evaluate and test our alignment model. SIDAR's pipeline begins with the construction of a 3D scene using Blender. The center of the plane is positioned at the origin of the coordinate system. The coordinates of the four corners of the plane are defined as follows:

$$X_1 = \begin{pmatrix} \frac{w}{2} \\ \frac{h}{2} \\ 0 \end{pmatrix}, X_2 = \begin{pmatrix} -\frac{w}{2} \\ \frac{h}{2} \\ 0 \end{pmatrix}, X_3 = \begin{pmatrix} \frac{w}{2} \\ -\frac{h}{2} \\ 0 \end{pmatrix}, X_4 = \begin{pmatrix} -\frac{w}{2} \\ -\frac{h}{2} \\ 0 \end{pmatrix} \tag{16}$$

Within this environment, multiple cameras can be positioned to capture the 2D plane from various perspectives. Translational adjustments enable the generation of image patches by capturing only specific regions of the plane, while rotational adjustments create stretched or skewed variations of the image. The geometric transformation between the image captured by the camera and the original plane is represented by a homography matrix, which is used for comparisons with our model's outputs 4.2. Each homography matrix $\mathbf{H}_\pi$ is defined as:

$$\mathbf{H}_\pi = \mathbf{K}\left(\mathbf{R} - \frac{1}{d}\mathbf{t}\mathbf{n}^\top\right)\mathbf{K}^{-1} \tag{17}$$

where $\mathbf{K}$ denotes the intrinsic camera matrix, $\mathbf{R}$ is the rotation matrix, $\mathbf{t}$ is the translation vector, $\mathbf{n}$ represents the normal vector to the plane and d indicates the distance between the camera and the plane.

To further enhance the realism of a synthetic dataset, diverse lighting conditions can be added into the scene. SIDAR supports various light sources, with customizable parameters including power and radius. Different lighting variations simulate different environmental conditions. Furthermore, 3D objects like circles, boxes, or cones can be added onto the scene to simulate occlusions in an image. These objects can be configured to cast shadows or obtain transparency, depending on the selected settings. The amount of occlusions, cameras and lightning spots are variable and can be adapted to fit the prefered complexity. Scenes can be generated in any countable quantity.

3 Approach

3.1 Planar Image Alignment

Our proposed model **MARF** is performing 2D image alignment and is using the 2D approach of BARF [8] as a base line. In the 2D approach of BARF we are dealing with one less dimension and therefore have to adjust the objective function accordingly: Instead of learning a full NeRF, our model is learning a homography transformation for each image. The transformation is mathematically represented by a warp function $W(x; p)$, where x corresponds to the pixel coordinates in the image, and p is a parameter vector

that defines the transformation. The image alignment task is framed as an optimization problem, where the goal is to minimize the photometric error between the predicted image I_{pred} and the reference images $\sum_{Input} I_{Input}$ $[I_1...I_N]$, where N represents each input image. This photometric error is quantified by the mean squared error (MSE) between the corresponding pixel intensities of the two images.
The optimization process begins with the objective function, which formalizes the goal of minimizing the photometric error. Our loss function can be defined as:

$$L_{rgb} = \sum_{N} \sum_{x} \|I_{pred}(W(x;p)) - I_N(x)\|_2^2 \tag{18}$$

This function seeks to minimize the MSE between the warped image $I_{pred}(W(x;p))$ and the target image $I_N(x)$, with the parameter vector p being iteractively adjusted to align the images as closely as possible.
To achieve this alignment, the warp update rule is applied to optimize the parameter vector p:

$$\Delta p = -A(x;p) \sum_{x} J(x;p)^\top \left(I_{pred}(W(x;p)) - I_N(x)\right) \tag{19}$$

Here, Δp represents the necessary adjustment to the parameter vector to improve alignment, with the steepest descent image $J(x;p)$ capturing the gradient information essential for this optimization process. The matrix $A(x;p)$, which depends on the chosen optimization algorithm, such as the Gauss-Newton method, influences how these parameters are updated. A crucial component of this process is the steepest descent image $J(x;p)$, defined as

$$J(x;p) = \frac{\partial I_{pred}(W(x;p))}{\partial W(x;p)} \cdot \frac{\partial W(x;p)}{\partial p} \tag{20}$$

where the warp function's Jacobian with regard to the parameters p and the image gradients of I_{pred} with respect to the warp function are combined. The image of the steepest descent has a major influence when determining the direction of the parameter updates in gradient-based optimisation.

3.2 Edge Alignment

Besides aligning the color values of each image, we compute edge-images from grayscale-transformed input images by using a combination of Sobel filtering [5] and Gaussian blurring. We introduce an extended loss function encorporating the L_{rgb} and an edge-alignment dependant loss value L_{Eedge}, where the weight between the two factors can be adjusted by a hyperparameter and also can change over time during the training process: Our model can value the L_{rgb} at the beginning of the model training more than the L_{Edge} or vice versa. Equally weighted losses are also possible.

We are generating edge-images as follows: Let $\mathbf{i}$ be a grayscale image. We first apply the Sobel filter to $\mathbf{i}$. The sobel filter consists of two kernels G_x and G_y

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad , G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{21}$$

which are then convoluted with the input-image to receive the image gradients S_x in x and S_y in y direction.

$$S_x = \mathbf{i} * G_x, \quad S_y = \mathbf{i} * G_y \tag{22}$$

Next,we calculate the magnitude of the gradient, which represents the edge-strength.

$$\mathbf{E} = \sqrt{S_x^2 + S_y^2} \tag{23}$$

To improve the alignment of the edges and making them more consistent to the original occlusion we blur the edge-image, and erode the mask-image and multiply them. This reduces the discrepancies in occlusion size and edge size.

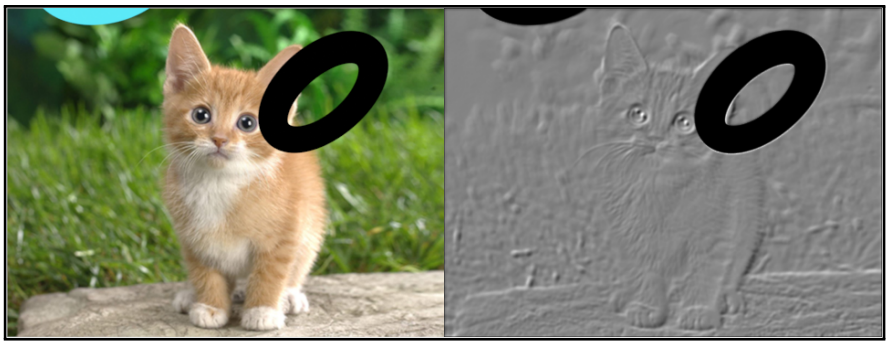


Figure 1: On the left side is the original input image and on the right side the final edge image

$$\hat{\mathbf{E}} = \mathbf{E} * \mathbf{G}_\sigma * \mathbf{M}_e \tag{24}$$

$\mathbf{G}_\sigma$ is a Gaussian kernel with a standard deviation σ. Our model uses a kernel size of 5×5 and sigmaX $= 0$, simplifying the operation to

$$\mathbf{G}_\sigma(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right) \tag{25}$$

and $\mathbf{M}_e$ as the eroded mask with the original mask $\mathbf{M}$ multiplied with a structuring element $\mathbf{B}$

$$\mathbf{M}_e = \mathbf{M} * \mathbf{B} \tag{26}$$

Finally, the inverse of the eroded mask will be multiplicated with $\hat{\mathbf{E}}$. A resulting edge-image can be seen in Figure 1. Using these edge-images, we can define our edge loss function as

$$L_{edge} = \left(\|E_{pred}(\mathbf{x}) - \hat{E}(\mathbf{x})\|_2^2\right) \tag{27}$$

$$L = (1 - \alpha) \cdot L_{rgb} + \alpha \cdot L_{edge} \tag{28}$$

α is a tunable hyperparameter that balances the influence of L_{rgb} and L_{edge} within the total loss. It can change over the course of iteration steps during the training process. This allows the model toprioritize L_{rgb} more heavily during certain phases of training and L_{edge} during others, depending on the needs of the model at different stages. Equally weighted losses are also possible. Explicitly, α is defined as:

$$\alpha = \theta_{\text{initial}} + (\theta_{\text{final}} - \theta_{\text{initial}}) \cdot \frac{\text{it}}{\text{max\_it}} \tag{29}$$
$$\text{with } \theta_{\text{initial}}, \theta_{\text{final}} \in [0, 1]$$

Here θ_{initial} is the initial value of α, θ_{final} is the final value of α. The current iteration is given by it while max_it is the maximum number of iterations.

3.3 Mask Generation

The input data of the network 2 is a combination of two multidimensional objects. The first is a self-defined grid to preserve spatial consistency, while the second is a vector extracted from the occlusion-filled image. Combined they determine the dimension of the input layer. There are 256 neurons by default in each of the three dense hidden layers that make up the network. The hidden layers employ the Rectified Linear Unit (ReLU) [10]activation function, because it can model non-linearities in the data without sacrificing the effectiveness of the training process. The function passes positive input values unchanged and sets negative values to zero, helping to minimise the problem of vanishing gradients and improve the network's learning

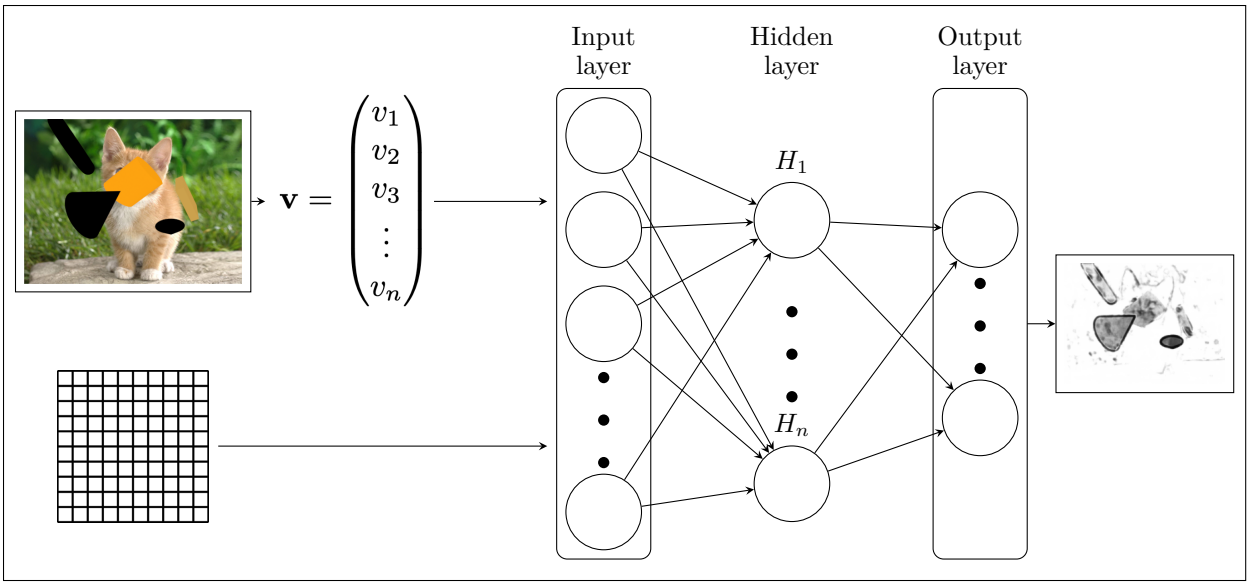


Figure 2: **Mask Generation Network** - The input to the MLP consists of a grid and a vector extracted from images. The network has three dense hidden layers and outputs the masks.

ability. The output layer consists of a sigmoid acitvation function [11]. Since the sigmoid function compresses the values to a range between 0 and 1, it is perfect for creating masks. Because it enables the model to create masks that exactly cover or hide portions of an image or other data structure, this compression is very crucial. The function is able to produce a continous spectrum rather than being binary.
The loss function 15 deals with the reconstruction error. This results in the combined loss function 30, which includes beside the mask all other functionalities described in 3.1 and 3.2.

$$L = (1 - \alpha) \cdot (M_x \cdot L_{rgb}) + \alpha \cdot (M_x \cdot L_{edge}) + \gamma \cdot (1 - M_x)^2 \tag{30}$$

The last term $(1 - M_x)^2$ is the mask loss, designed to ensure that the regions of interest in the rendered images align with the target masks.

4 Experimental Results

Following the description of our methodology we will look into quantitative and visual results. Similar to BARF, we are using the image of a cat from ImageNet [2]. Using SIDAR, we create five image patches out of that groundtruth image and feed it into our network. The goal of these experiments is to see if our model is improving on BARF and Ha-NeRF by using quantitative metrics. The experiments are using the same settings as BARF: The MLP contains four 256-dimensional hidden layers and is trained for 5000 iterations with a learning rate of 0.001. The BARF bundle-adjustment feature is applied for the first 2000 iterations. The Mask Generation MLP is used as described in 3.3. For the mask generation process we trained the model for 6000 iterations and reduced the learning rates for the image and homography MLP's of BARF to 0.0001 instead.

4.1 Datasets

Using SIDAR (see 2.4) we created five datasets with different obstacles to test our models robustness. Each dataset consists of a batch of five images. The datasets are numbered from Batch 1 to Batch 5, with Batch 1, Batch 3 and Batch 4 shown in Figure 3 containing masks for occlusions in addition to the RGB images. As shown in Figure 4 Batch 2 und Batch 5 contain only the generated images.

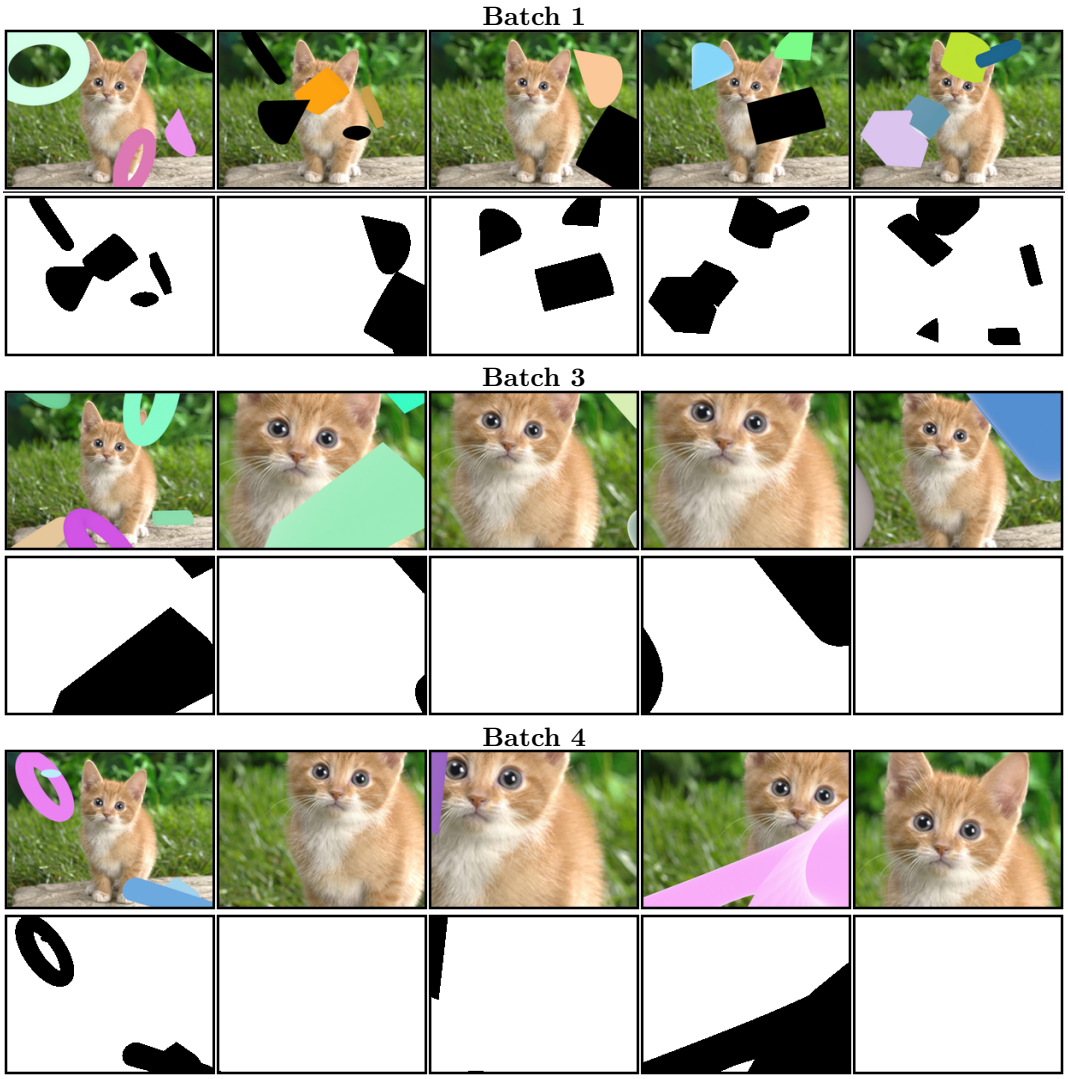


Figure 3: Batch 1 is using occlusions on the original ground truth image. Batch 3 combines rotational transformation and occlusions. Batch 4 combines occlusions and translational transformation for patches.

Figure 4: Batch 2 contains smaller patches of the original image captured through pure translation. Batch 5 uses the ground truth image with different lightning conditions

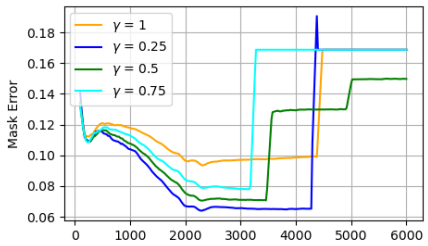
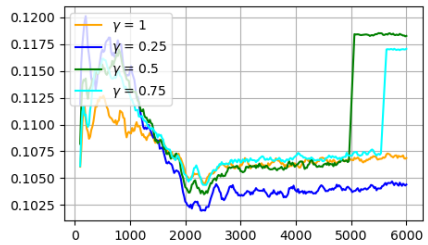
Batch 1	**Lowest** M_{err}	
$\beta = 0.25$	0.065	
$\beta = 0.5$	0.071	
$\beta = 0.75$	0.078	
$\beta = 1$	0.09	
Batch 3	**Lowest** M_{err}	
$\beta = 0.25$	0.1047	
$\beta = 0.5$	0.1065	
$\beta = 0.75$	0.1170	
$\beta = 1$	0.1070	

Table 1: Mask error over time - For these tests, we have used an alpha value initially starting at 0 and increasing to 1 (see Chapter 3.2) The blue curve denoting a loss weight of $\gamma = 0.25$ achieves the lowest errors for both datasets. It can be observed though, that a sudden increase of the error value happens for some curves at different timesteps.

4.2 Quantitative Metrics

The metrics we use for the following experiments are the Peak signal-to-noise ratio defined as

$$PSNR = -10 \cdot \log_{10}(L_{rgb}) \tag{31}$$

which is evaluating the reconstruction of the original image. Using this definition, a higher value denotes a better reconstruction. Additionally, we are comparing the estimated homographies $\hat{H}$ (previously denoted as W in Chapter 3.1) with the groundtruth homographies H to compute the estimation error as

$$H_{err} = \left\| \frac{H}{\sqrt[3]{\det(H)}} - \frac{\hat{H}}{\sqrt[3]{\det(\hat{H})}} \right\|_2^2 \tag{32}$$

4.3 Mask Generation

As described in 3.3, mask generation happens during the training of the model. For making the quality of our masks measurable, we are employing the metric

$$M_{err} = \|M - M_{pred}\|_2^2 \tag{33}$$

where M signifies the ground truth mask and M_{pred} the predicted mask of our model. Our results are shown in Table 1 and the visual results for Batch 1 can be seen in Figure 5. Although the masks initially are not set, our model manages to correctly align the cat. There are still issues with the actual masking of occlusions as there are still fragments of the occlusions visible for Batch 1.

In Table 1 we can observe a sudden increase in M_{err}. This can be explained with our used loss function: When we are generating masks, the balancing factor for the mask generation is applied to the loss value as well. The closer our model gets to a correct alignment, the higher also our balancing loss is in comparison to our L_{rgb} and L_{edge}. Our model therefor removes the masks alltogether for a temporary decrease in the overall loss value, but due to that change the L_{rgb} and L_{edge} are increasing once again. Therefor, depending on the used dataset, the generation process needs to stop at different iteration times to successfully generate a mask.

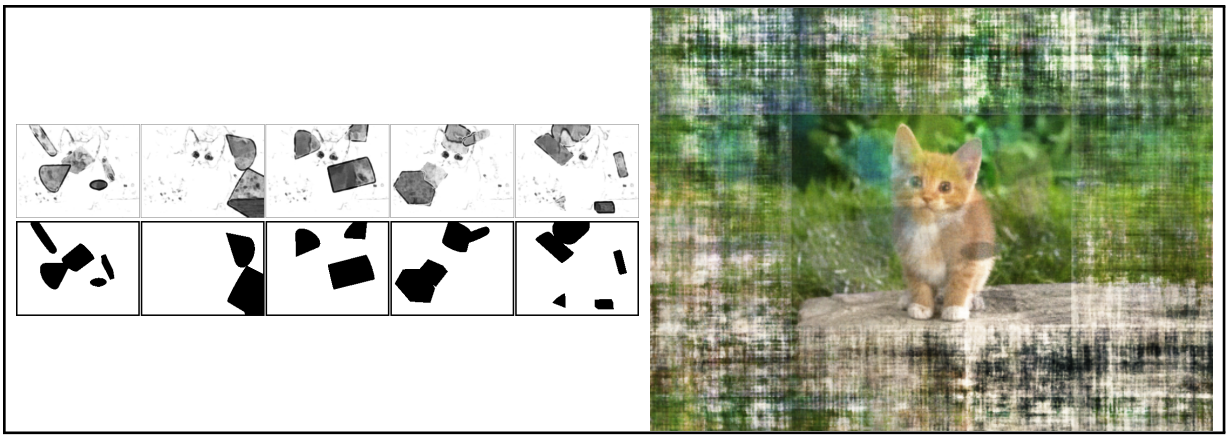


Figure 5: The visual results of our mask generation for Batch 1. The masks cover most of the expected elements but also other unrelated elements such as the eyes of the cat. The generated result does still contain occlusions because the masks are not occluding all pixels to 100 percent as suggested by the gray color of the masks.

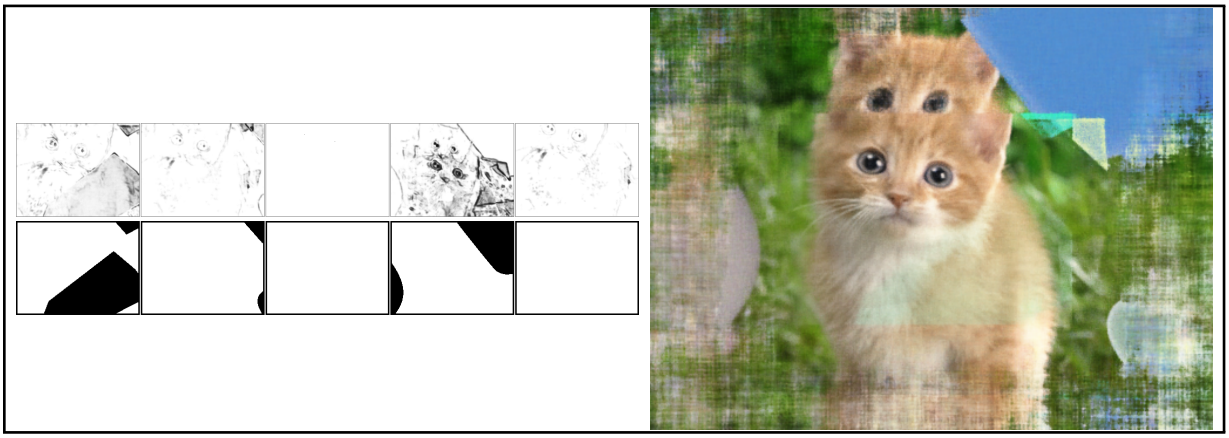


Figure 6: The visual results of our mask generation for Batch 3. The masks do not seem to be generated correctly, especially if we compare the masks for the fourth image of Batch 3. The aligned image contains a lot more fragments of occlusions.

The mask generation results for Batch 3 can be seen in Figure 6. The dataset contains perspective changes and therefor our model has a harder time to estimate a correct mask. Additionally, not every element of the ground truth image can be found in each used image patch, reducing the similarities of each image. Overall the alignment as well as the generated mask are unsatisfying.

4.4 Alignment Results

The following experiments used the pre-generated masks from SIDAR as described in 4.1. For these experiments we are not trying to generate masks, therefore the weight is $\gamma = 0$ for all further experiments. The experiments focused on the necessity of our introduced features, namely occlusion removal and edge alignment. The results are accumulated in Table 2. Table 3 is displaying the result images and metric value development for Batch 3. It can be seen that our model using all features achieves significantly better results in comparison to BARF according to our metrics, especially when encountering maskable occlusions within our datasets. In particular, we observed that any approach not utilizing occlusion removal leads to significantly worse results both quantitatively and visually speaking. Therefore, we omitted any case that is not using that feature, except the base BARF model. What can also be observed in the case of Batch 2 is that the edge alignment functionality leads to a better homography estimation particularly if applied at the end of the alignment process. Generally speaking, our model, depending on its configuration, performs better

	Batch 1	Batch 2		Batch 3		Batch 4		Batch 5
	$PSNR \uparrow$	$PSNR \uparrow$	$H_{err} \downarrow$	$PSNR \uparrow$	$H_{err} \downarrow$	$PSNR \uparrow$	$H_{err} \downarrow$	$PSNR \uparrow$
BARF	16,638	26,230	2,806	20,744	1,676	23,123	2,653	27,036
MARF, $\alpha = 0$	34,674	26,230	2,806	24,872	1,547	24,753	1,768	27,036
MARF, $\alpha = 0.5$	34,315	29,548	5,854	25,281	1,689	24,728	1,774	26,991
MARF, $0 \xrightarrow{\alpha} 1$	34,067	26,288	2,768	24,967	1,591	24,707	1,742	27,082
MARF, $1 \xrightarrow{\alpha} 0$	35,396	29,729	5,849	25,422	1,726	24,684	1,760	27,226

Table 2: Comparison of PSNR and Homography Error H_{err} across different batches and between BARF and our model configurations. There is no Homography Error for Batch 1 and Batch 5 due to the fact that these sets do not contain any camera translations. An orange cell denotes the worst result of a column, green the best result. Higher PSNR values or lower Homography Error values signify a better result. The α value is either fixed, or changing depending of the iteration as described in Chapter 3.2.

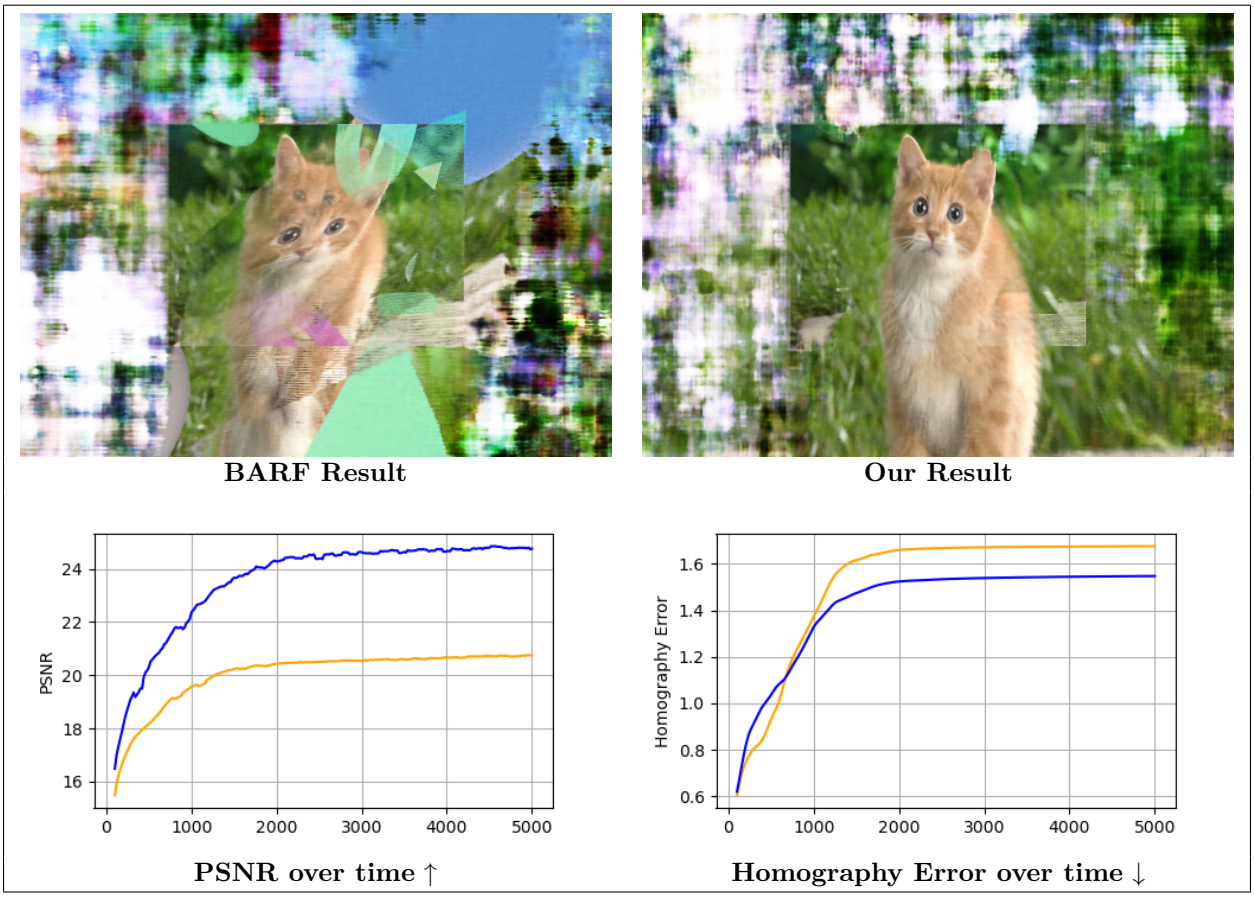


Table 3: Batch 3 Results Comparison - The orange curve is denoting BARFs performance per metric, the blue one is denoting our best performing configuration of MARF (see Table 2).

than BARF according to our metrics. There is still room for improvement when it comes to illumination changes, as in e.g. Batch 5. More result Images as well as graphs can be found in this paper's appendix 6.

5 Conclusion

Given the experimental results discussed before, we can prove that our proposed model is an improvement on top of the underlying BARF model. However, there is still room for future improvements.

Particularly the mask generation needs to be adapted further. While it already generates good masks for datasets without any perspective changes, generally it is not delivering sufficient results. Additionally, while it makes sense to learn masks during an alignment, the image alignment itself will always suffer through the initial lack of a complete and correct mask. Therefore in the current state of our mask generation process, the generated masks should rather be used as a starting configuration of a consecutive alignment attempt instead of being taken as a ground truth mask. Future research could focus on that aspect of our model.

Due to the usage of edge alignment, our model delivers more robust image alignments than BARF. Especially in cases where slight illumination changes or shadows are occuring, this edge alignment leads to better image alignments (Table 2, Batch 5). Still, there is room for improvement when it comes to more significant illumination issues in images. Future research could focus on further preprocessing of the input images or the implementation of shadow removal as a part of the MLP functioning as the implicit image function. Additionally, encorporating keypoint alignment into the MLP or loss calculation could possibly be an improvement for situations where shadows or light are leading to difficulties in the alignment process.

Since keypoints tend to be stable between two images, they also could be used to pre-align images before starting the alignment process, therefore generating a starting configuration for the homographies of each image patch that can be further refined during the alignment process. This could help to prevent our model to align to a (wrong) local minimum which would correspond to a wrong or non-ideal image alignment.

References

[1] Xingyu Chen et al. "Hallucinated neural radiance fields in the wild". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, pp. 12943–12952.

[2] Jia Deng et al. "ImageNet: A large-scale hierarchical image database". In: *2009 IEEE Conference on Computer Vision and Pattern Recognition*. 2009, pp. 248–255. DOI: 10.1109/CVPR.2009.5206848.

[3] Ian J. Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[4] Richard Hartley and Andrew Zisserman. *Multiple View Geometry in Computer Vision*. 2nd ed. New York, NY, USA: Cambridge University Press, 2003. ISBN: 0521540518.

[5] Nick Kanopoulos, Nagesh Vasanthavada, and Robert L Baker. "Design of an image edge detection filter using the Sobel operator". In: *IEEE Journal of solid-state circuits* 23.2 (1988), pp. 358–367.

[6] Monika Kwiatkowski, Simon Matern, and Olaf Hellwich. "SIDAR: Synthetic Image Dataset for Alignment & Restoration". In: *Proceedings of the 19th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications*. SCITEPRESS - Science and Technology Publications, 2024. DOI: 10.5220/0012391400003660. URL: http://dx.doi.org/10.5220/0012391400003660.

[7] Yong Li et al. "Transformer for object detection: Review and benchmark". In: *Engineering Applications of Artificial Intelligence* 126 (2023), p. 107021. ISSN: 0952-1976. DOI: https://doi.org/10.1016/j.engappai.2023.107021. URL: https://www.sciencedirect.com/science/article/pii/S0952197623012058.

[8] Chen-Hsuan Lin et al. "BARF: Bundle-Adjusting Neural Radiance Fields". In: *IEEE International Conference on Computer Vision (ICCV)*. 2021.

[9] Ben Mildenhall et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis". In: *ECCV*. 2020.

[10] Vinod Nair and Geoffrey E Hinton. "Rectified linear units improve restricted boltzmann machines". In: *Proceedings of the 27th international conference on machine learning (ICML-10)*. 2010, pp. 807–814.

[11] P. J. Werbos. "Applications of Advances in Nonlinear Sensitivity Analysis". In: *System Modeling and Optimization: Proc. IFIP*. Ed. by R. Drenick and F. Kozin. Springer, 1982.

6 Appendix

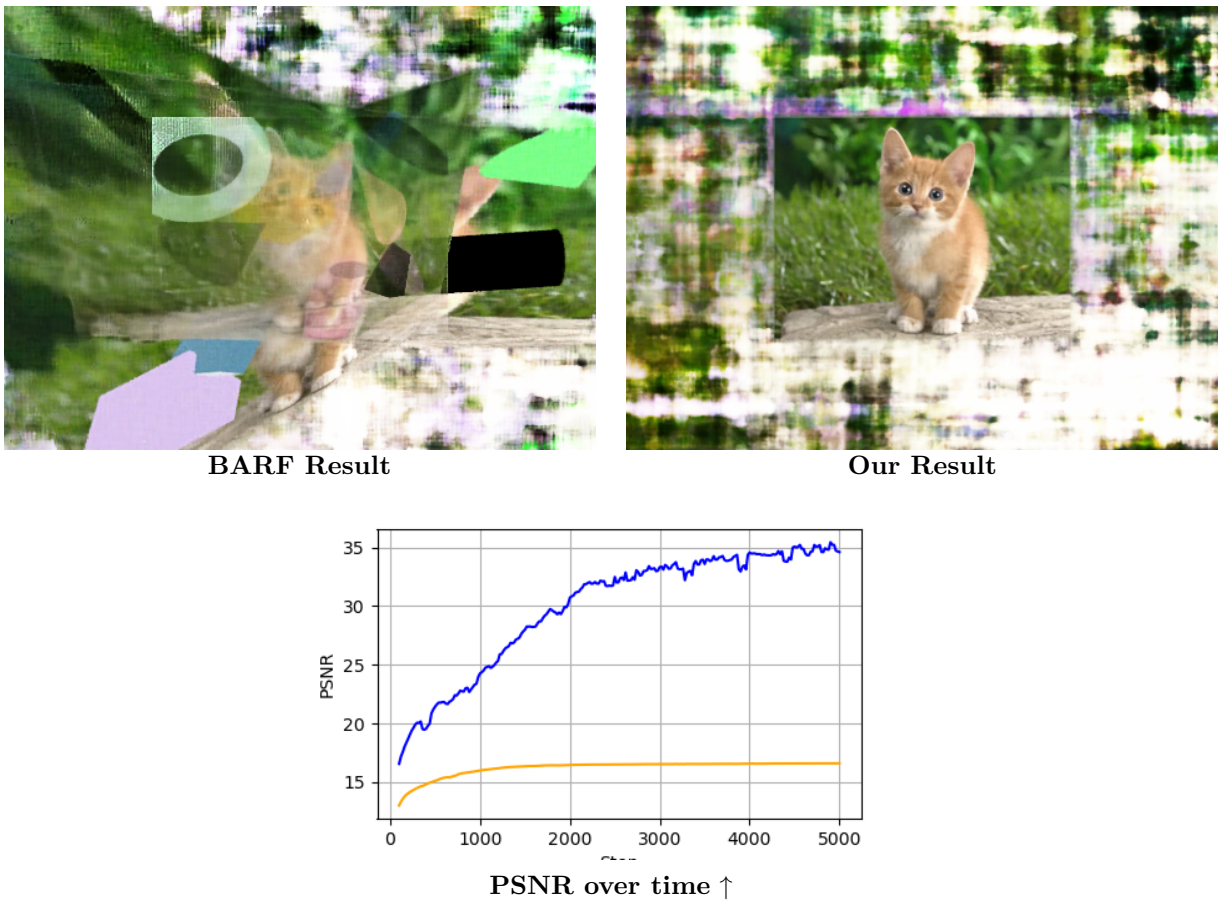


Table 4: Batch 1 Results Comparison - The orange curve is denoting BARFs performance per metric, the blue one is denoting our best performing configuration of MARF (see Table 2).

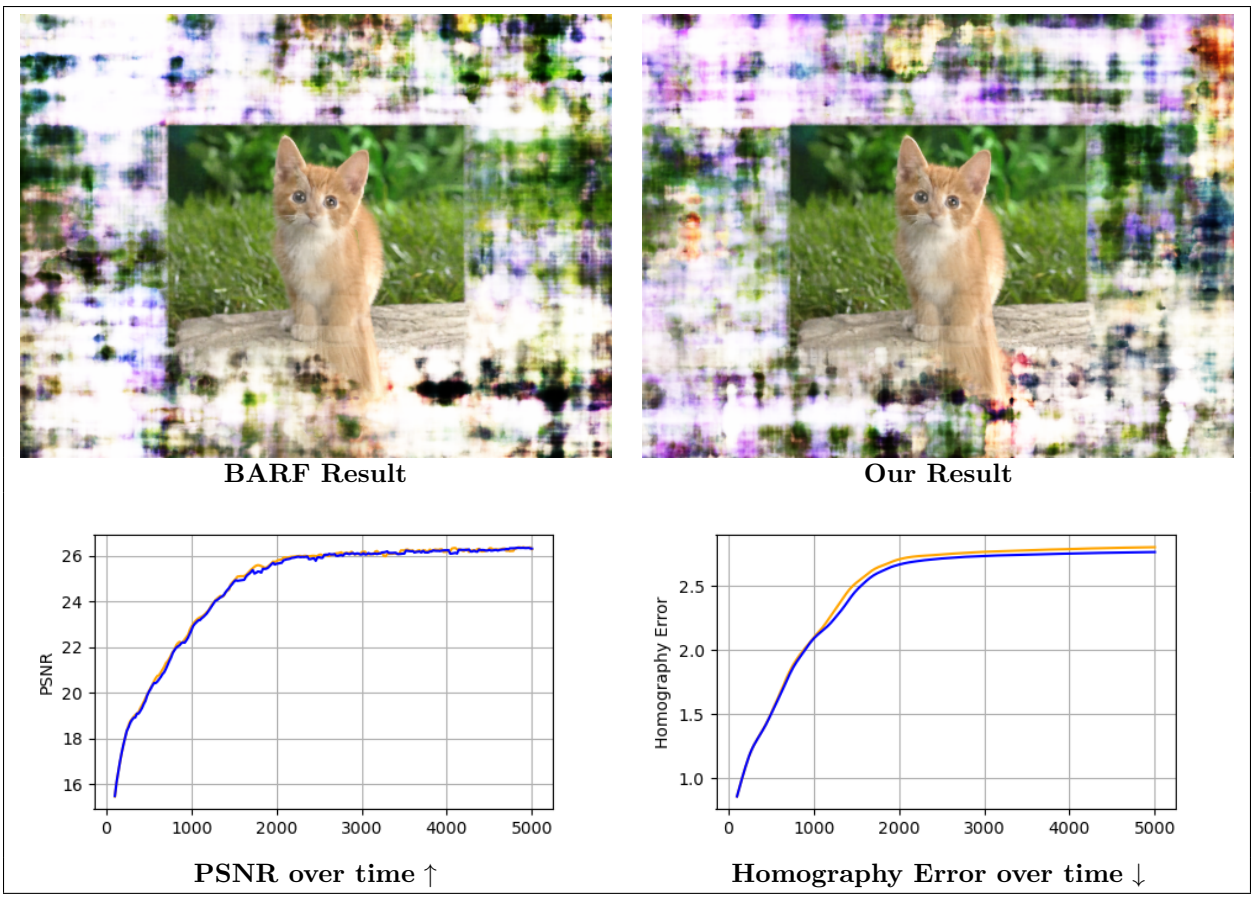


Table 5: Batch 2 Results Comparison - The orange curve is denoting BARFs performance per metric, the blue one is denoting our best performing configuration of MARF (see Table 2).

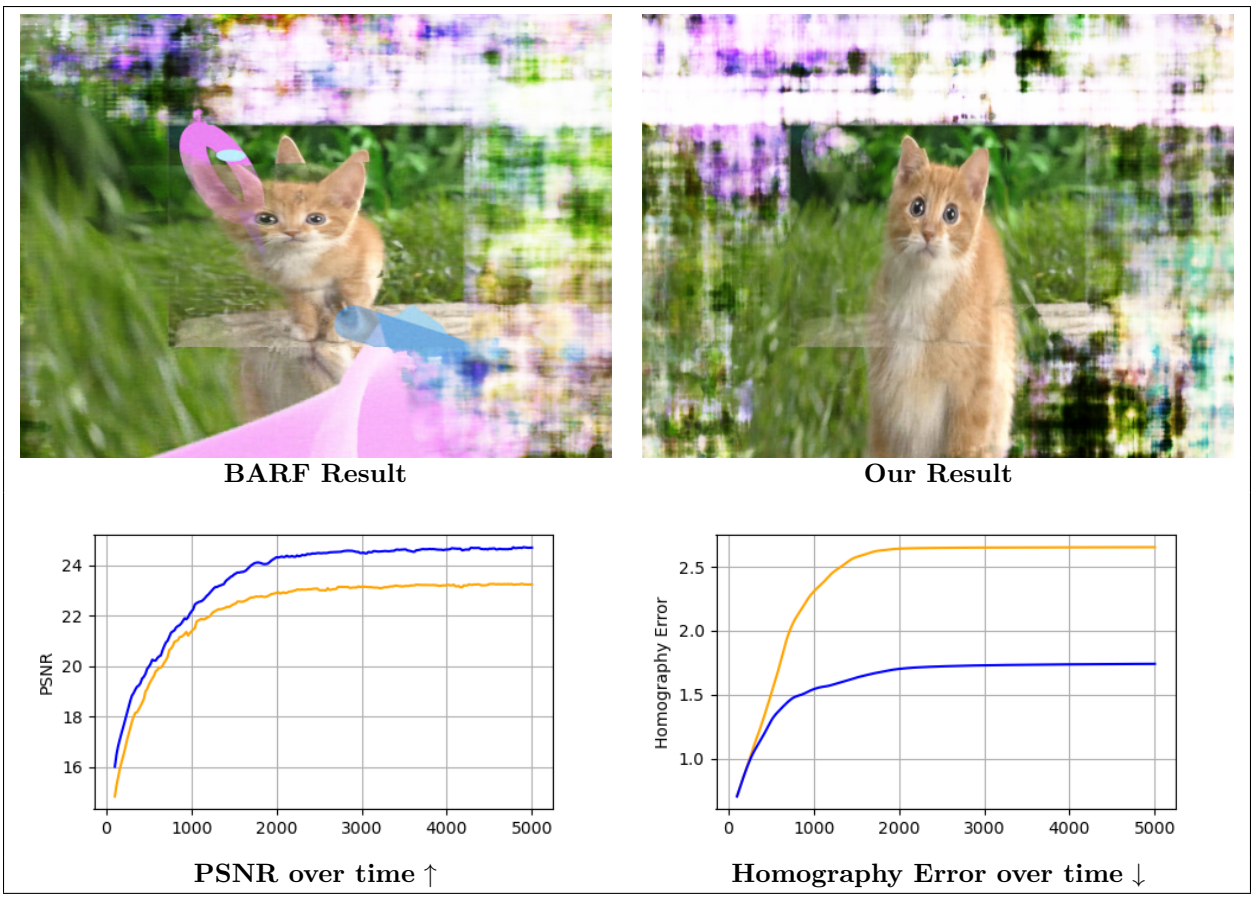


Table 6: Batch 4 Results Comparison - The orange curve is denoting BARFs performance per metric, the blue one is denoting our best performing configuration of MARF (see Table 2).

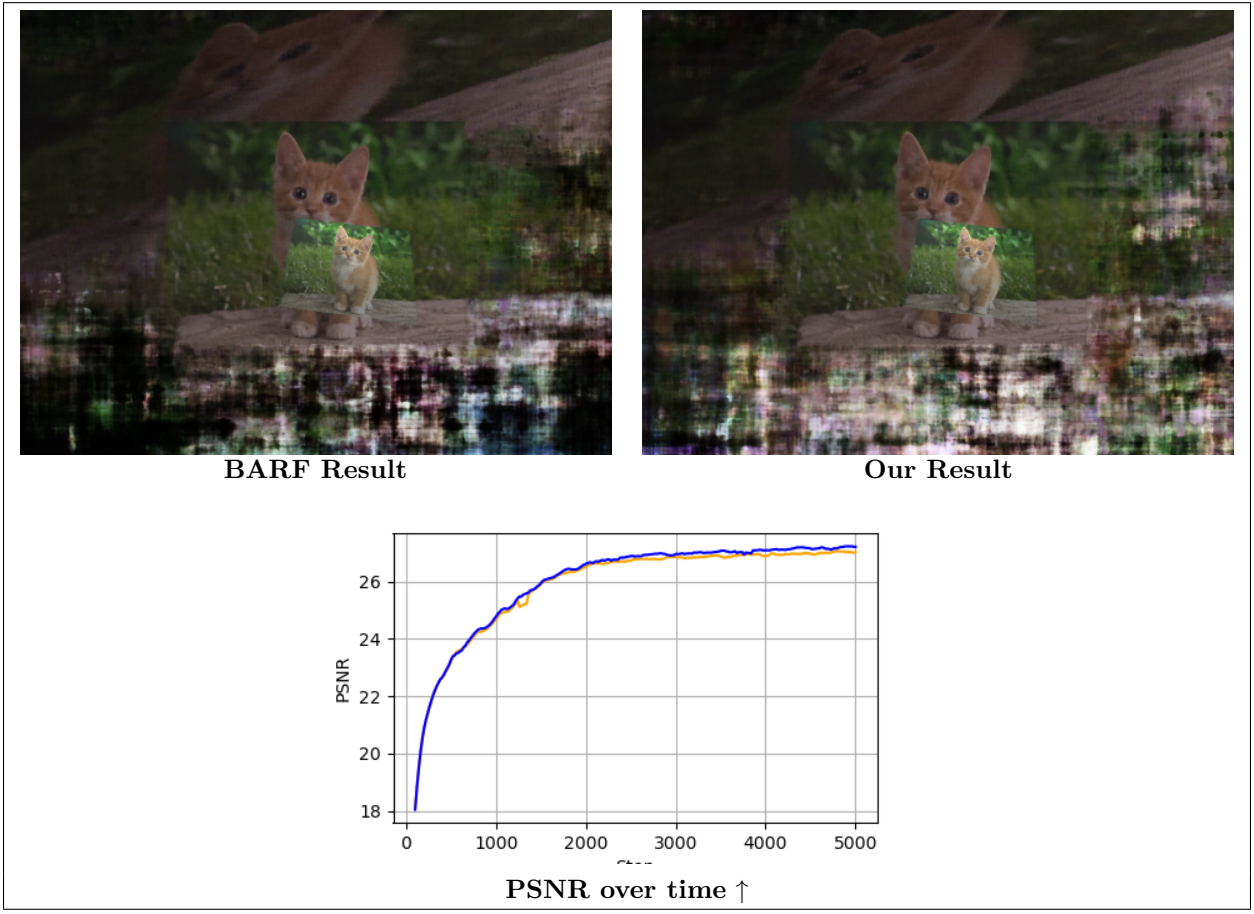


Table 7: Batch 5 Results Comparison - The orange curve is denoting BARFs performance per metric, the blue one is denoting our best performing configuration of MARF (see Table 2).